



Slides for P3740R1

Last chance to fix `std::nontype`

Document number:

P3753R1

Date:

2025-06-20

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/nontype-slides.cow



IMPORTANT: This document has custom controls:

-  ,  : go to the next slide
-  ,  : go to previous slide

Last chance to fix `std::nontype` P3740R1

Introduction

- P2472R3 added `std::nontype` constructors for `std::function_ref`
- merged into P0792R14, plenary-approved Varna 2023 for C++26
- `std::nontype` possibly obsoleted by `std::constant_wrapper` (P2781R9)
 - Wording review completed by LWG
- "non-type template parameter" term was removed by P2841R1 (C++26)

Conclusion

History is messy. `std::nontype` may be a bad feature now, or at least have a terrible name.

std::function_ref refresher

`std::function_ref` is a "function wrapper" similar to `std::function`, `std::move_only_function`, `std::copyable_function`, but without ownership.

`std::function_ref<R(Args...) cv noex>` stores:

- *bound-entity* — *BoundEntityType* (basically a `cv void*`)
- *thunk_ptr* — `R(*) (BoundEntityType, Args...) noex`



NOTE: For illustration purposes, we'll ignore `cv`, `noex`, `invoke`.

std::nontype in std::function_ref

```
template<class F>          function_ref(F*);
template<class F>          function_ref(F&&);
template<auto f>           function_ref(nontype_t<f>);
template<auto f, class U> function_ref(nontype_t<f>, U&&);
template<auto f, class T> function_ref(nontype_t<f>, T*);
```

```
template<auto f>
function_ref(nontype_t<f>) {
    this->bound-entity = {};
    this->thunk_ptr = [] (BoundEntityType, Args&&... args) -> R {
        return f(std::forward<Args&&>(args)...);
    };
}
```

With `std::constant_wrapper`

```
template<class F>          function_ref(F*);  
template<class F>          function_ref(F&&);  
template<auto f>           function_ref(constant_wrapper<f>);  
template<auto f, class U>   function_ref(constant_wrapper<f>, U&&);  
template<auto f, class T>   function_ref(constant_wrapper<f>, T*);
```

```
template<auto f>  
function_ref(constant_wrapper<f>) {  
    this->bound-entity = {};  
    this->thunk_ptr = [] (BoundEntityType, Args&&... args) -> R  
    { return constant_wrapper<f>::value  
      (std::forward<Args&&>(args)...); };  
}
```

Alternatives considered

- Low consensus:
 - `std::stateless` constructor tag
 - remove `std::nontype`, revisit in C++29
 - wait for P1045R1 `constexpr` function parameters
 - expose the *BoundEntityType*
- Plausible:
 - replace `std::nontype` with `std::constant_wrapper`
 - rename `std::nontype` (to `std::constant_arg`)

std::constant_wrapper concerns

std::cw<&free_function> already behaves *much* like wrapped function:

- F&& and F* constructors are candidates
 - not a problem, these lose in overload resolution
- danger of inconsistent behavior with std::move_only_function etc.

```
constexpr int f() { return 0; }  
std::move_only_function<int()> r = std::cw<f>; // OK ?!
```

- **Solution:** Provide corresponding overloads for *all* function wrappers.
- **Urgency:** If done later, meaning of existing code subltly changed.